Introductory Computational Fluid Dynamics (CHE-462)
Complex Engineering Problem

Grading
Obtained Marks:
Total Marks:

Submitted By

Name	Registration No.
Haseeb Tahir	**11-3-1-015-2022**
Saeedullah Khan	**11-3-1-037-2022**

Department of Chemical Engineering
Pakistan Institute of Engineering and Applied Sciences
P. O. Nilore, Islamabad, Pakistan

Table of Contents

List of Figures

Abstract

This study investigates and compares various numerical methods for solving the one-dimensional transient heat conduction problem in a long, slender metallic rod of length 20 cm. The rod is initially at a uniform temperature of 25°C, while its ends are subjected to time-dependent boundary conditions given by (T(0,t) = 80 + 2t) and (T(L,t) = 40 + 0.5t). The numerical methods analyzed include the Explicit Method (FTCS), Implicit Method (BTCS), Crank–Nicolson Method, and Richardson Method.

The thermophysical properties of the material, including thermal conductivity, density, and specific heat capacity, are used to compute the thermal diffusivity, which governs heat propagation within the rod. MATLAB is employed to simulate the temperature distribution along the rod over time.

The performance of each method is evaluated based on stability, accuracy, and computational efficiency. The results demonstrate that explicit methods require strict stability conditions, while implicit methods provide unconditional stability. Among all methods, the Crank–Nicolson scheme offers the best balance between accuracy and stability. This comparative analysis provides deeper insight into the applicability of numerical techniques for transient heat conduction problems.

Keywords: transient heat conduction, finite difference methods, stability, numerical analysis, one-dimensional heat transfer

Problem Statement

A long, slender metallic rod of length **20 cm** is considered for one-dimensional transient heat conduction analysis. The rod is initially at a uniform temperature of **25°C** throughout its length.

The ends of the rod are subjected to time-dependent boundary conditions such that:

- Left end: **$T(0, t) = 80 + 2t$ °C**
- Right end: **$T(L, t) = 40 + 0.5t$ °C**

where time **t** is in seconds.

Given Data

Thermal conductivity:
$k = 0.49$ cal/(s·cm·°C)

Specific heat capacity:
$C = 0.2174$ cal/(g·°C)

Density:
$\rho = 2.7$ g/cm³

Thermal Diffusivity

$\alpha = k / (\rho C)$

Discretization Parameters

$\Delta x = 2$ cm
$\Delta t = 0.1$ s

Stability Factor (Explicit Method)

$\lambda = (\alpha \times \Delta t) / (\Delta x)^2$

Objectives

The methods considered are:

- Explicit Method (FTCS)
- Implicit Method (BTCS)
- Crank–Nicolson Method
- Richardson Method

Methods Implementation

Explicit Method (FTCS)

The Forward-Time Central-Space (FTCS) method is an explicit finite difference technique in which the temperature at the next time level is computed directly from known values at the current time level. It is simple and computationally efficient but conditionally stable.

The governing discretized equation is:

$$T_i^{n+1} = T_i^n + \lambda(T_{i-1}^n - 2T_i^n + T_{i+1}^n)$$

where,

$$\lambda = \frac{\alpha \Delta t}{(\Delta x)^2}$$

Stability Condition:

$$\lambda \leq 0.5$$

If this condition is violated, the solution becomes unstable and diverges.

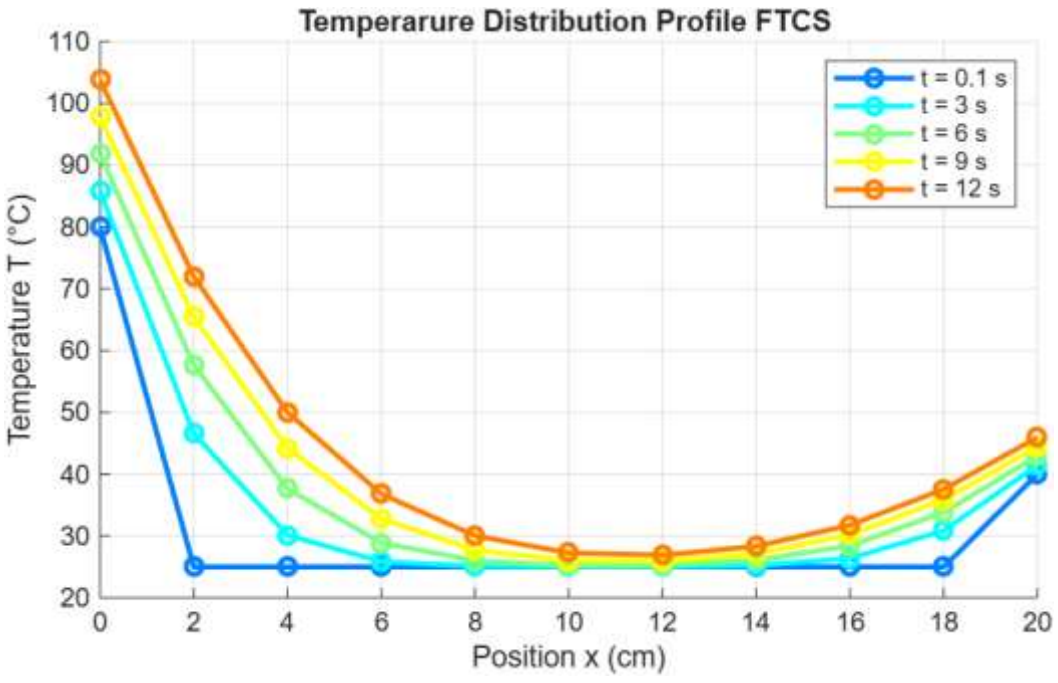

Figure 1 Temperature distribution using FTCS method

Implicit Method (BTCS):

The Backward-Time Central-Space (BTCS) method is an implicit scheme in which future temperature values are computed by solving a system of linear equations.

The discretized equation is:

$$\frac{T_i^{n+1} - T_i^n}{\Delta t} = \alpha \frac{T_{i+1}^{n+1} - 2T_i^{n+1} + T_{i-1}^{n+1}}{(\Delta x)^2}$$

This method is:

- Unconditionally stable
- More computationally expensive (requires matrix solution)

Results

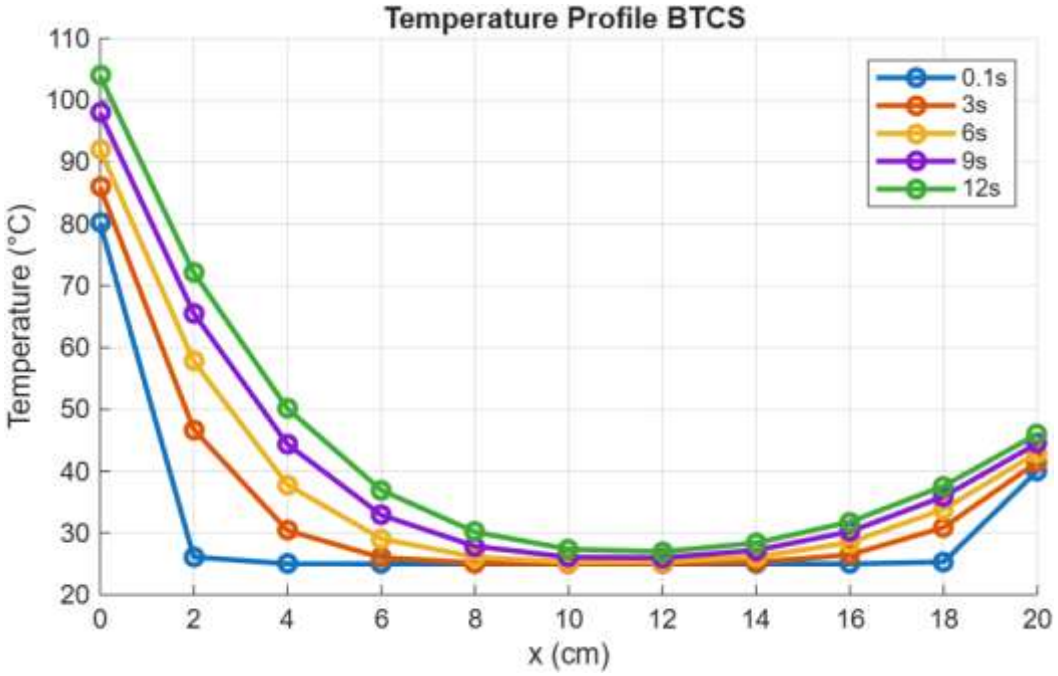

Figure 2 Temperature distribution using BTCS method

Crank-Nicholson Method:

The Crank–Nicolson method is a semi-implicit scheme that combines FTCS and BTCS, providing second-order accuracy in both time and space.

$$\frac{T_i^{n+1} - T_i^n}{\Delta t} = \frac{\alpha}{2(\Delta x)^2}[(T_{i+1}^{n+1} - 2T_i^{n+1} + T_{i-1}^{n+1}) + (T_{i+1}^n - 2T_i^n + T_{i-1}^n)]$$

Key Features:

- Unconditionally stable
- Higher accuracy than FTCS and BTCS
- Requires solving linear systems

This is the best method overall for this problem.

Results

It is unconditionally stable, meaning it does not have a strict requirement for small time steps to maintain stability.

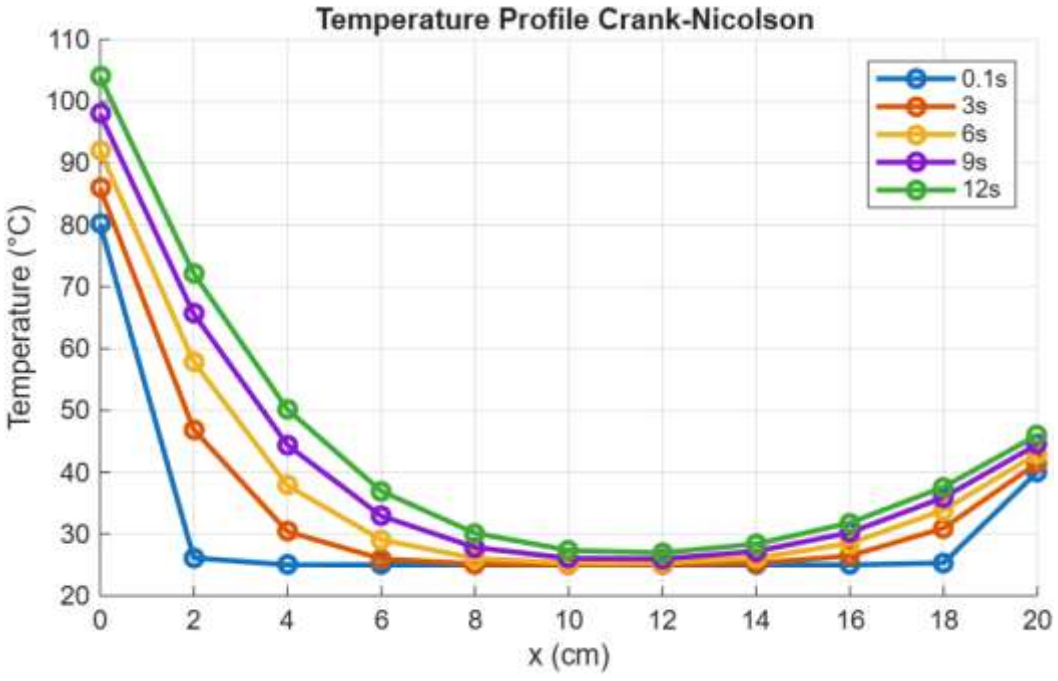

Figure 3 Temperature distribution using Crank-Nicholson method

Ricardson Method:

The Richardson method is an explicit second-order method in time, given by:

$$\frac{T_i^{n+1} - T_i^{n-1}}{2\Delta t} = \alpha \frac{T_{i+1}^n - 2T_i^n + T_{i-1}^n}{(\Delta x)^2}$$

Important Behavior:

- Second-order accurate
- Unconditionally unstable for heat conduction problems

Even with small time steps, errors grow rapidly, leading to unrealistic results.

Results

Richardson being an explicit algorithm so there is stability issues associated with it which can be visualized in figure below. As time step (Δt) increases the temperature profile turns unrealistic.

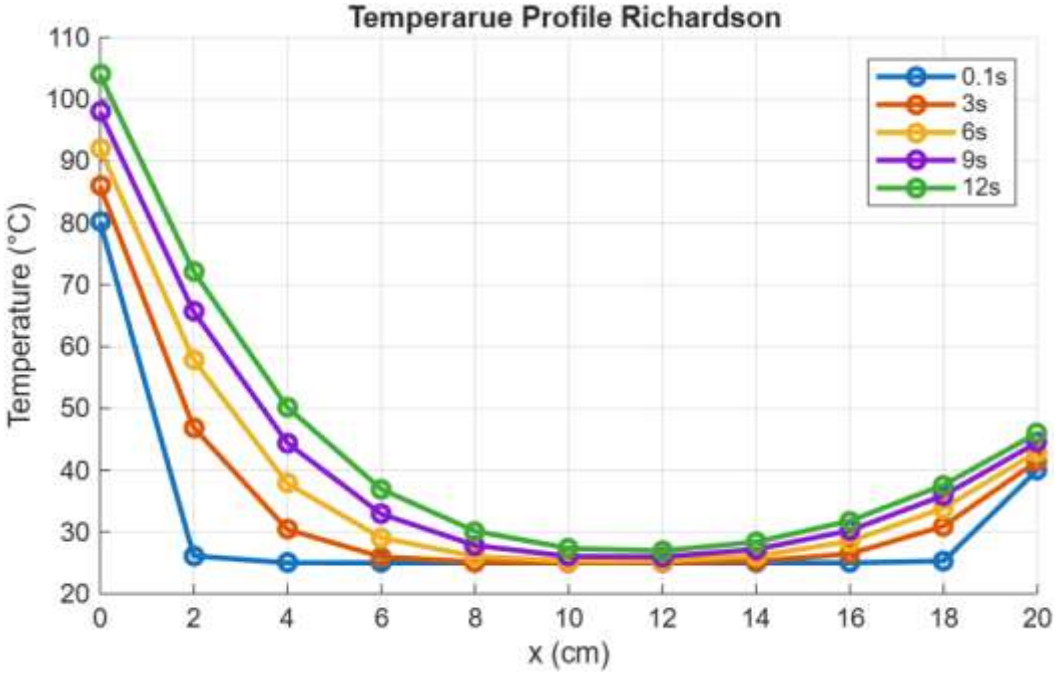

Figure 4 Temperature distribution using Richardson method

Discussion

The numerical results obtained from the four methods highlight clear differences in stability and accuracy. The FTCS method, although simple and computationally efficient, requires strict adherence to the stability condition $\lambda \leq 0.5$. When satisfied, it produces reasonable results; otherwise, the solution diverges.

The BTCS method demonstrates unconditional stability, making it reliable for larger time steps. However, it requires solving a system of equations at each time step, increasing computational cost.

The Crank–Nicolson method provides superior accuracy due to its second-order discretization in both time and space. It maintains stability for all time steps while producing smooth and physically consistent temperature profiles. This makes it the most balanced and preferred method for transient heat conduction problems.

The Richardson method, despite its higher-order formulation, exhibits instability for parabolic equations like heat conduction. The results show that errors grow rapidly with time, leading to non-physical temperature distributions.

Overall, the comparison indicates that while explicit methods are useful for quick computations, implicit and semi-implicit methods are more reliable for accurate and stable solutions.

Conclusion

The comparative study of numerical methods for transient heat conduction demonstrates that each method has distinct advantages and limitations. The FTCS method is simple and fast but conditionally stable. The BTCS method ensures unconditional stability but requires higher computational effort. The Crank–Nicolson method emerges as the most effective approach, offering both high accuracy and stability. In contrast, the Richardson method is unsuitable for this problem due to its inherent instability.

Therefore, for practical engineering applications involving transient heat conduction, the Crank–Nicolson method is recommended as the optimal choice.

Appendix A

Code snippets of all methods are provided here.

Explicit Method (FTCS)

```
clear; clc; close all;
%% PARAMETERS
L = 20;              % Length of rod (cm)
dx = 2;              % Spatial step (cm)
Nx = L/dx + 1;       % Number of nodes

% Material properties
k = 0.49;            % Thermal conductivity (cal/s·cm·°C)
rho = 2.7;           % Density (g/cm^3)
C = 0.2174;          % Specific heat (cal/g·°C)

% Thermal diffusivity
alpha = k / (rho * C);

dt = 0.1;            % Time step (s)
lambda = alpha * dt / dx^2;   % Stability factor

% Stability check
if lambda > 0.5
   warning('FTCS method unstable! Reduce dt or increase dx.');
end

%% GRID
x = linspace(0, L, Nx);
```

```
%% TIME SETTINGS
t_final = 12;
Nt = round(t_final / dt);

%% INITIAL CONDITION (T = 25°C everywhere)
T = 25 * ones(1, Nx);

%% BOUNDARY CONDITIONS
T_left  = @(t) 80 + 2*t;
T_right = @(t) 40 + 0.5*t;

%% STORAGE
T_all = zeros(Nt+1, Nx);
T_all(1,:) = T;

%% TIME LOOP
for n = 1:Nt

    t = (n-1)*dt;
    T_new = T;

    % Apply boundary conditions
    T_new(1)   = T_left(t);
    T_new(end) = T_right(t);

    % FTCS  (interior nodes)
    for i = 2:Nx-1
        T_new(i) = T(i) + lambda * (T(i+1) - 2*T(i) + T(i-1));
```

```
    end

    % Update
    T = T_new;
    T_all(n+1,:) = T;
end

%% PLOTTING
times_to_plot = [0.1, 3, 6, 9, 12];

figure;
hold on;

colors = jet(length(times_to_plot));
legend_entries = {};

for j = 1:length(times_to_plot)
    n_plot = round(times_to_plot(j)/dt) + 1;
    plot(x, T_all(n_plot,:), '-o', 'Color', colors(j,:), 'LineWidth', 2);
    legend_entries{end+1} = ['t = ' num2str(times_to_plot(j)) ' s'];
end

xlabel('Position x (cm)');
ylabel('Temperature T (°C)');
title('FTCS Solution (Updated Problem)');
legend(legend_entries, 'Location', 'best');
grid on;
hold off;
```

Implicit Method (BTCS)

```
clear; clc; close all;
%% PARAMETERS
L = 20;
dx = 2;
dt = 0.1;
k = 0.49;
rho = 2.7;
C = 0.2174;
alpha = k/(rho*C);

x = 0:dx:L;
n = length(x);

t_end = 12;
time = 0:dt:t_end;

r = alpha*dt/dx^2;

%% INITIAL CONDITION
T = 25 * ones(n, length(time));

%% BOUNDARY CONDITIONS
for k_t = 1:length(time)
   T(1,k_t)   = 80 + 2*time(k_t);
   T(end,k_t) = 40 + 0.5*time(k_t);
end
```

```
%% COEFFICIENT MATRIX
A = (1 + 2*r)*eye(n-2);
for i = 1:n-3
    A(i,i+1) = -r;
    A(i+1,i) = -r;
end

%% TIME LOOP
for k_t = 1:length(time)-1

    b = T(2:end-1,k_t);

    % Boundary effects (implicit uses k+1)
    b(1)   = b(1) + r*T(1,k_t+1);
    b(end) = b(end) + r*T(end,k_t+1);

    T(2:end-1,k_t+1) = A\b;
end

%% PLOTTING
times_to_plot = [0.1, 3, 6, 9, 12];
figure; hold on;

for i = 1:length(times_to_plot)
    idx = round(times_to_plot(i)/dt)+1;
    plot(x, T(:,idx), '-o', 'LineWidth', 2);
end
```

```
xlabel('x (cm)');
ylabel('Temperature (°C)');
title('BTCS Method (Updated Problem)');
legend('0.1s','3s','6s','9s','12s');
grid on
```

Crank-Nicholson Method

```
clear; clc; close all;
%% PARAMETERS
L = 20; dx = 2; dt = 0.1;
k = 0.49; rho = 2.7; C = 0.2174;
alpha = k/(rho*C);

x = 0:dx:L;
Nx = length(x);

t_final = 12;
Nt = t_final/dt;

lambda = alpha*dt/dx^2;

%% INITIAL CONDITION
T = 25*ones(Nx,1);

%% BC FUNCTIONS
T_left  = @(t) 80 + 2*t;
T_right = @(t) 40 + 0.5*t;

%% MATRICES
```

```
a = -lambda/2;
b = 1 + lambda;
c = -lambda/2;

A = diag(b*ones(Nx-2,1)) + diag(a*ones(Nx-3,1),-1) + diag(c*ones(Nx-3,1),1);
B = diag((1-lambda)*ones(Nx-2,1)) + diag((lambda/2)*ones(Nx-3,1),-1) +
diag((lambda/2)*ones(Nx-3,1),1);

%% STORAGE
times_to_plot = [0.1,3,6,9,12];
plot_idx = round(times_to_plot/dt);
T_store = zeros(length(plot_idx), Nx);

count = 1;

%% TIME LOOP
for n = 1:Nt

    t_now = n*dt;
    t_prev = (n-1)*dt;

    T(1) = T_left(t_now);
    T(end) = T_right(t_now);

    RHS = B*T(2:end-1);

    RHS(1) = RHS(1) + (lambda/2)*(T_left(t_now) + T_left(t_prev));
    RHS(end) = RHS(end) + (lambda/2)*(T_right(t_now) + T_right(t_prev));
```

```
    T(2:end-1) = A\RHS;

    if ismember(n, plot_idx)
        T_store(count,:) = T';
        count = count + 1;
    end
end

%% PLOT
figure; hold on;
for i = 1:length(times_to_plot)
    plot(x, T_store(i,:), '-o','LineWidth',2);
end

xlabel('x (cm)');
ylabel('Temperature (°C)');
title('Crank-Nicolson Method');
legend('0.1s','3s','6s','9s','12s');
grid on;
```

Richardson Method

```
% Richardson Method (Updated)

clear; clc; close all;

%% PARAMETERS
L = 20; dx = 2; dt = 0.1;
k = 0.49; rho = 2.7; C = 0.2174;
alpha = k/(rho*C);
```

```
lambda = alpha*dt/dx^2;

x = 0:dx:L;
nx = length(x);

t_end = 12;
nt = t_end/dt;

%% INITIAL CONDITION
T = 25*ones(nx, nt+1);

time = (0:nt)*dt;

%% BOUNDARY CONDITIONS
T(1,:)   = 80 + 2*time;
T(end,:) = 40 + 0.5*time;

%% RICHARDSON LOOP
for n = 1:nt

    T_half = zeros(nx,1);

    % Half-step BC
    T_half(1)   = (T(1,n) + T(1,n+1))/2;
    T_half(end) = (T(end,n) + T(end,n+1))/2;

    % Predictor
```

```
    for i = 2:nx-1
        T_half(i) = T(i,n) + (lambda/2)*(T(i-1,n) - 2*T(i,n) + T(i+1,n));
    end

    % Corrector
    for i = 2:nx-1
        T(i,n+1) = T(i,n) + lambda*(T_half(i-1) - 2*T_half(i) + T_half(i+1));
    end
end

%% PLOT
times_to_plot = [0.1,3,6,9,12];

figure; hold on;

for i = 1:length(times_to_plot)
    idx = round(times_to_plot(i)/dt)+1;
    plot(x, T(:,idx), '-o','LineWidth',2);
end

xlabel('x (cm)');
ylabel('Temperature (°C)');
title('Richardson Method');
legend('0.1s','3s','6s','9s','12s');
grid on;
```